

Roméo et Python 3 - by JL

0) Introduction à ce mini-guide pour ROMEO :

- Pourquoi ce guide ?
 - ROMEO est le centre de calcul de l'université de Reims Champagne-Ardenne, c'est un espace numérique partagé sur lequel de nombreux utilisateurs lancent des calculs très gourmands. Il convient donc de l'utiliser avec précaution pour que tout fonctionne bien.
 - En plus, étant confiné à cause du COVID-19, j'ai eu le temps de tester quelques travaux et de rédiger ces quelques lignes...
- Ressources d'aide sur Roméo et sur Python3 :
 - Roméo :
 - [page officielle](#)
 - [techno. centre](#)
 - Python 3 :
 - [CSEstack](#)
 - [Doc. officielle](#)

1) Roméo :

1.a) Connexion à Roméo (depuis le Mac)

- Pour plus de sécurité, il est préférable de se connecter à ROMEO via le client VPN Cisco de l'URCA. Ensuite, on se connecte à la plateforme avec un terminal (il faut remplacer username dans tout le document par son propre login) :

```
% ssh -o ForwardX11=yes -o ForwardAgent=yes -o ForwardX11Trusted=yes username@romeo  
ologin1.univ-reims.fr
```

- On arrive sur sa page d'accueil :

```
Last login: Tue Apr 21 08:14:13 2020 from 86.25.20.68
```

```

                                $
                                &
                                &&#
                                &&#%#%#%#&&#
                                ###((&&
                                (((    (((//((    (((&&
                                (((    (((//***//    (((&&i
                                (((    (((//**, **//    &&&
                                ###    (((//***//    &&&
                                ##%#    (((//((    %%%
                                %%%    /(    %%%
                                &&&&&    %%%
                                &&
                                https://romeo.univ-reims.fr
                                &&

```

Bienvenue sur le supercalculateur ROMEO - NG

Pour recevoir une notification en cas de mise à jours des notes de maintenance, abonnez vous à la liste romeo-maintenance: <https://listes.univ-reims.fr/sympa/info/romeo-maintenance>

- * La nouvelle machine (ROMEO_2018) est fonctionnelle. L'équipe technique installe les logiciels à la demande : merci de mettre un ticket. Une partie de l'ancienne machine est en maintenance.
- * 31/10/19 : La calculateur ROMEO est opérationnel.
- * 17/04/20 : Le noeud RomeoLogin2 est actuellement hors service. Merci d'utiliser RomeoLogin1

```
[username@romeologin1 ~]$
```

- On arrive sur le shell bash dans lequel on peut taper des commandes.

1.b) Version de Python

```
$ python --version
Python 2.7.5
```

1.c) Charger le module Python3 (+deeplearning)

```
$ module load python/3.6.9_gnu-deeplearning
$ python --version
Python 3.6.9
```

1.d) Connaître la liste des packages installés sous Python3

- Première solution (ne fonctionne pas toujours : Bus error ?) :

```
$ ipython
Python 3.6.9 (default, Sep 16 2019, 17:10:04)
Type 'copyright', 'credits' or 'license' for more information
IPython 7.8.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: help("modules")

...
```

- Deuxième solution :

```
$ pip list
```

```
[username@romeologin1 ~]$ pip list
```

Package	Version
-----	-----
absl-py	0.8.0
astor	0.8.0
attrs	19.1.0
backcall	0.1.0
bash-kernel	0.7.2
bleach	3.1.0
certifi	2019.9.11
...	
scikit-image	0.15.0
scikit-learn	0.21.3
scipy	1.3.1
seaborn	0.9.0
segmentation-models	1.0.1
Send2Trash	1.5.0
setuptools	40.6.2
six	1.12.0
sklearn	0.0
tensorboard	1.14.0
tensorflow	1.14.0
tensorflow-estimator	1.14.0
tensorflow-gpu	1.14.0
termcolor	1.1.0
terminado	0.8.2
testpath	0.4.2
Theano	0.7.0
torch	1.2.0
torchvision	0.4.0
tornado	6.0.3
traitlets	4.3.2
urllib3	1.25.3
wcwidth	0.1.7
webencodings	0.5.1
Werkzeug	0.15.6
wheel	0.33.6
widgetsnbextension	3.5.1
wrapt	1.11.2
xgboost	0.90

You are using pip version 18.1, however version 20.1b1 is available.

You should consider upgrading via the 'pip install --upgrade pip' command.

1.e) Commandes Linux utiles :

- Savoir où l'on se trouve :

```
$ pwd
/home/username
```

- Lister le contenu d'un dossier :

```
$ ls -l
total 12
drwxrwxr-x. 8 username username 4096 Apr 21 08:48 bases
-rw-rw-r--. 1 username username  44 Apr 21 09:40 moduleload.txt
drwxrwxr-x. 3 username username 4096 Apr 21 09:39 python
```

- Créer un dossier :

```
$ mkdir projet1
$ ls -l
total 16
drwxrwxr-x. 8 username username 4096 Apr 21 08:48 bases
-rw-rw-r--. 1 username username  44 Apr 21 09:40 moduleload.txt
drwxrwxr-x. 2 username username 4096 Apr 21 10:09 projet1
drwxrwxr-x. 3 username username 4096 Apr 21 09:39 python
$
```

- Se déplacer dans les dossiers (chemins relatifs) :

- descendre :

```
$ cd projet1
$ pwd
/home/username/projet1
```

- remonter :

```
$ cd ..
$ pwd
/home/username
```

1.f) Editeur de textes disponible par défaut : vi (the best editor ever)

```
$ vi toto.py
```

- Quelques commandes utiles de vi :

- "h" : aller à gauche
- "j" : descendre d'une ligne

- "k" : remonter d'une ligne
- "l" : aller à droite
- "x" : effacer le caractère courant
- "dd" : effacer la ligne courante
- "i" : insérer du texte (mode insertion)
- "ESC" : sortir du mode insertion
- ":w" : écrire le fichier
- ":q" : quitter vi
- ":wq" : écrire le fichier et quitter vi

2) Python3 :

2.a) Encodage des fichiers :

- Mettre sur la première ligne du fichier :

```
# coding: utf-8
```

2.b) Ecrire du code dans un fichier python :

- On prend un exemple de code (ici "ip3.py") qui va lire toutes les images et les écrire dans un fichier 'images' à partir d'une liste listImages :

```

# coding: utf-8
import numpy
import skimage
import time
import glob
import ldm
import pickle

path = '../bases/coil-100'
dirs=glob.glob(path)
listImages=[]

for dir in dirs:
    dir=dir+'/*'
    files=glob.glob(dir)

    t = time.time()

    for file in files:
        tmpList=[]
        tmpList.append((dir[-3:-2]))
        tmpList.append((file))
        tmpList.append(skimage.img_as_ubyte(skimage.io.imread(file)))
        listImages.append(tmpList)

with open('images', 'wb') as fp:
    pickle.dump(listImages, fp)

    elapsed = time.time() - t
print('Elapsed time downloading images : %f s.' % elapsed)

print('Shape: ')
print(numpy.shape(listImages[1][2]))

print('type: %s' % type(listImages[1][2]))

print('dtype: %s' % listImages[1][2].dtype)

nb = len(listImages)
print('Number of images: %d' % nb)

```

3) Utilisation de ROMEO :

3.a) Lancement de code avec ROMEO

- **ATTENTION**, on ne lance jamais de processus gourmand en mémoire avec le serveur de login : on

peut le bloquer et perturber le travail des autres collègues !

- Il faut donc écrire un SCRIPT sbatch qui va demander des ressources et démarrer lorsque ROMEO aura du temps machine à lui accorder.
- Un exemple de script sbatch (de nom "go_0001.sbatch") pour démarrer notre script "ip3.py" (vu en 2.b ci-dessus) :

```
#!/bin/bash
#SBATCH --comment "Lecture d'images"
#SBATCH -J "Lecture_0001"

#SBATCH --error=job.%J.err
#SBATCH --output=job.%J.out

#SBATCH -p short
#SBATCH --time=00:30:00

#SBATCH --mem 20000
#SBATCH --mem-per-cpu 2000

module load python/3.6.9_gnu-deeplearning
python python/ip3.py
```

- Ce script possède les champs suivants :
 - un commentaire : pour indiquer ce que fait le script
 - un nom : pour identifier le script
 - un fichier erreur : pour lire les éventuelles erreurs
 - un fichier de sortie : pour voir les éventuels résultats affichés (avec la fonction print)
 - une priorité : ici short (car on veut passer le script rapidement : il n'est pas très gourmand en temps de calcul)
 - un temps d'exécution maximum estimé : cela va de paire avec la priorité juste au-dessus
 - une quantité mémoire totale : ici 20 Go
 - une quantité mémoire par CPU : ici 2 Go
 - On lance ensuite les deux commandes du script : on charge le module python3 avec traitement d'images et on lance notre script qui est dans le dossier python par rapport au dossier racine (chemin relatif)
 - il y a tout un tas de commandes pour réserver un certain nombre de processeurs, allouer des ressources supplémentaires (GPU...), il faut lire la documentation (voir un peu plus loin en 3.b).
- On lance ensuite ce script sbatch avec la commande sbatch :

```
[username@romeologin1 ~]$ sbatch go_0001.sbatch
Submitted batch job 1892785
```


- Le script passe dans la queue d'exécution visible avec la commande squeue :

```
[username@romeologin1 ~]$ squeue | grep username
      1892785          short Lecture_ username PD           0:00          1 (None)
```

- Le script, une fois exécuté produit deux fichiers que l'on peut voir avec la commande :

```
[username@romeologin1 ~]$ more job.1892785.out
[username@romeologin1 ~]$ more job.1892785.err
Traceback (most recent call last):
  File "python/ip3.py", line 29, in <module>
    elapsed = time.time() - t
NameError: name 't' is not defined
[username@romeologin1 ~]$
```

- Bon, là il y a une erreur (bel exemple de ce qu'il ne faut pas faire) car j'ai utilisé une variable `t` non définie (car l'indentation du code est mauvaise, c'est ma faute).
- Mais quand ça marche, le fichier d'erreur ne contient rien et le fichier de sortie contient les affichages (print) du résultat.
- **DONC** il vaut mieux avoir un code qui tourne bien (à tester en local sur sa machine avec une partie des données) **AVANT** d'envoyer sur ROMEO pour un gain de temps non négligeable.

3.b) Le techno-centre ROMEO :

- Nous avons accès à la documentation en ligne de ROMEO sur notre espace utilisateur ROMEO :



Techno-Centre

Fiche utilisateur

Mes projets

Mon utilisation

Déconnexion

- Accueil
- MarkDown
- romeoBOX
- Historique
- Supercalculateur ROMEO (2013)
- Serveur QLM
- Notes de maintenance
- URCloud
- Supercalculateur ROMEO (2018)
 - Pour débuter
 - Les Modules
 - Top500 Green500 et HPCG

Soumission de jobs

Editer Historique Renommer Toutes les pages + ^

Technocentre / Supercalculateur ROMEO (2018) / Pour débuter / Soumission de jobs

Le gestionnaire de job Slurm est disponible, en version 16.05.1. Sa documentation complète est disponible :

<https://slurm.schedmd.com/archive/slurm-16.05.1/>

- Le site officiel de SLURM : <http://slurm.schedmd.com>
- Une rapide introduction à SLURM : <https://computing.llnl.gov/tutorials/slurm/slurm.pdf>
- Les tutoriels proposés par l'éditeur du logiciel : <http://slurm.schedmd.com/tutorials.html>
- Le manuel des commandes SLURM : http://slurm.schedmd.com/man_index.html

Principales commandes :

Voici la liste des principales commandes pour *Slurm* :

```
squeue
sinfo
squeue -u *login*
scontrol show job 1234
sbatch
scancel 1234
scancel --interactive --user=*login*
sprio
sprio -l
srun
salloc
sacct
```

Affichage de la liste des jobs
 Etats des partitions (classes) et des serveurs
 Affichage de la liste des jobs pour un utilisateur particulier
 Affichage du détail sur mon job 1234
 Soumission d'un job via un script de soumission
 Annulation d'un job (ou 1234 est le numéro du job)
 Annulation de tous les jobs d'un utilisateur
 Affichage de la priorité relative des jobs en attente
 Affichage plus complet de la priorité
 Exécution immédiate d'une commande
 Lancement d'un job avec shell interactif
 Interroger les informations d'accounting

Script de soumission

Pour lancer un job, il convient d'écrire un script qui contient deux parties :

- Les instructions slurm sont écrites au début du fichier de soumission, et commencent par *#SBATCH*. Elles décrivent les besoins de votre job
- Les commandes à exécuter au démarrage du job Il convient ensuite de soumettre le script à l'aide de la commande *sbatch*

- Il est **VIVEMENT** recommandé de lire la partie Soumission de Jobs afin d'éviter de se faire bloquer son compte par les administrateurs de ROMEO !
- Bon courage, portez-vous bien, amitiés, Jérôme.